TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN · BASES DE DATOS II

TRABAJO PRÁCTICO N.º 10 (UNIDAD 8): FOOD STORE EN LA FERIA

La base de la tablet y el cierre de la capa de datos

Alumno: Máximo Juan Cruz Quiroga | **Docente:** Sergio Neira | UTN FRM 2026

Consigna 1 · La base de la tablet

1.a Qué datos viajan a la tablet

Para garantizar la continuidad operativa del food truck en ferias o eventos sin conexión a Internet, la tablet móvil almacena una copia local en **SQLite 3 (modo STRICT)**. Sin embargo, no todos los datos del servidor central deben viajar al dispositivo móvil. La siguiente tabla sintetiza la partición de atributos entre el servidor y el cliente:

Columna / Atributo	¿Viaja?	Columna en SQLite	Justificación Técnica y Criterio de Selección
id_producto	**SÍ**	producto_cache.id	Clave primaria obligatoria para identificar unívocamente cada ítem al armar borradores de pedido.
nombre	**SÍ**	producto_cache.nombre	Requerido en la interfaz para que el vendedor visualice qué comanda está cargando.
precio_actual	**SÍ**	producto_cache.precio_centavos	Viaja multiplicado por 100 como entero para eliminar imprecisiones de coma flotante en SQLite.
stock	**SÍ**	producto_cache.stock_visto	Permite al operador advertir anticipadamente al comensal si el stock está próximo a agotarse.
activo	**SÍ**	disponible / eliminado	Baja lógica: los productos descontinuados (activo=false) se ocultan del catálogo disponible.
fecha_copia	**SÍ**	producto_cache.fecha_copia	Marca de tiempo para auditar la frescura del catálogo recibido desde PostgreSQL.
id_categoria	NO	—	Innecesario en la venta rápida en feria; minimiza el payload del CSV y simplifica las consultas.
usuarios / passwords	NO	—	**Mínimo Privilegio:** Nunca deben viajar credenciales ni hashes de contraseñas a terminales móviles.
clientes / correos	NO	—	**Privacidad:** La venta al paso en feria no requiere acceso al padrón general de clientes registrados.
pedidos / ventas	NO	—	El historial contable reside exclusivamente en PostgreSQL y MongoDB; la tablet solo gestiona borradores.

Regla de Oro: «El móvil propone, el servidor dispone»

En la arquitectura de Food Store, **PostgreSQL es la única e indiscutible fuente de verdad**. La base de datos SQLite en la tablet opera como una caché transaccional temporal de solo lectura para el catálogo y un almacén de borradores locales. La aplicación móvil jamás descuenta stock en el servidor ni confirma ventas firmes: el estado SIMULADO_APROBADO es una pre-validación matemática local. Los precios finales, la deducción efectiva de existencias y la facturación fiscal son potestad exclusiva y centralizada de PostgreSQL.

Consigna 2 · Sincronizar y resolver

2.c Simulación y Verificación de los Casos C2 a C6 del Semáforo

Se prepararon escenarios intencionales en una copia de pruebas de PostgreSQL, exportando una segunda instantánea (servidor_snapshot_2c.csv) donde el producto 1 (Hamburguesa Clásica) subió de $4.500 a $4.800 y el producto 3 (Papas Fritas) sufrió baja lógica. El algoritmo del semáforo (semaforo.sql) arrojó los siguientes resultados:

Caso	UUID Solicitud	Situación Preparada	Estado Semáforo	Motivo Registrado / Resolución
C2-a	c2-solicitud-1	Borrador con precio anterior ($4.500) frente a nuevo precio ($4.800).	**CONFLICTO**	Cambió el precio observado en centavos.
C2-b	c2-solicitud-2	El cliente acepta el precio: se actualiza borrador a $4.800.	**SIMULADO_APROBADO**	Precio, disponibilidad y stock coinciden plenamente.
C3	c3-solicitud	Borrador por 99 unidades (stock real en servidor es 35).	**RECHAZADO**	Stock insuficiente para satisfacer la cantidad demandada.
C4	c4-solicitud	Borrador con producto 3, dado de baja lógica (disponible=0).	**RECHAZADO**	Producto no disponible / dado de baja.
C5	c5-solicitud	Borrador con catálogo actualizado sin discrepancias.	**SIMULADO_APROBADO**	Precio, disponibilidad y stock coinciden.
C6	c5-solicitud (bis)	Reintento de reenvío con el mismo identificador UUID.	**RECHAZO DE PK**	UNIQUE constraint failed: solicitud_uuid duplicada.

Demostración de Inmutabilidad en PostgreSQL (verificar_sin_cambios_2c.sql)

Al finalizar las pruebas del semáforo en SQLite, se ejecutó la consulta de auditoría en la base central de PostgreSQL: SELECT COUNT(*) FROM pedidos; y SELECT id_producto, stock FROM productos;. **El resultado fue contundente: cero filas añadidas y stock intacto**. La tablet jamás alteró los datos del servidor central.

Idempotencia y Protección Real en Sistemas Productivos (Parada 8)

En un entorno productivo real, la protección probada en el Caso C6 debe residir en la **capa API transaccional de PostgreSQL**. Al recibir un paquete de sincronización, el backend debe registrar el solicitud_uuid dentro de una tabla de idempotencia bajo una transacción serializable. Si el UUID ya fue procesado, el servidor descarta el procesamiento duplicado y retorna de inmediato el comprobante previamente emitido, evitando descontar inventario dos veces o generar doble cobro al cliente.

Consigna 3 · Integrar y revisar

3.a Mapa de la Capa de Datos Unificada (Los Tres Motores de Food Store)

Food Store consolida su arquitectura de datos mediante tres motores de persistencia especializados, articulados según sus ventajas operativas y patrones de acceso:

Motor de Base de Datos	Base / Colección	¿Qué datos almacena?	Patrón de Acceso	¿Manda? (Autoridad)
PostgreSQL 17 ***(Relacional)***	foodstore_dev	Catálogo maestro, precios de lista, stock real, clientes, usuarios, auditoría y pedidos confirmados.	OLTP Transaccional con ACID estricto y claves foráneas.	**SÍ (Fuente de Verdad). Manda sobre catálogo, inventario y facturación.**
MongoDB 7+ / 9+ ***(NoSQL Documental)***	foodstore_nosql .pedidos	Historial de pedidos cerrados con líneas embebidas y snapshot comercial inmutable del cliente.	Lectura masiva (Single-Document Read) sin JOINs para apps móviles.	**NO.** Es una copia derivada; ante cualquier discrepancia prevalece PostgreSQL.
SQLite 3.50+ ***(Embebido Móvil)***	foodstore_movil .db	Caché local del catálogo (precios en centavos), borradores de comanda y cola del semáforo.	Lectura y escritura local offline en la feria sin conexión a red.	**Solo sobre sus borradores locales** mientras no se hayan transmitido.

Matriz de Autoridad y Flujo de Información entre Motores

Dato / Atributo de Negocio	Motor que Decide (Autoridad)	Presencia en MongoDB (TP9)	Presencia en SQLite (Tablet Móvil)
Precio Vigente	PostgreSQL (productos.precio_actual)	No almacenado (solo guarda precio histórico)	precio_centavos (Precio visto temporal)
Stock en Tiempo Real	PostgreSQL (productos.stock)	No interviene	stock_visto (Control offline preliminar)
Baja Lógica	PostgreSQL (productos.activo)	No interviene	disponible y eliminado (0 o 1)
Venta Confirmada	PostgreSQL (pedidos y detalles)	Sí, como documento histórico cerrado	**Nunca. La tablet solo emite borradores**
Precio de Venta Histórico	PostgreSQL (detalles_pedido.precio_unitario)	Sí (lineas[].precio_unitario)	precio_centavos_visto (en borrador)
Datos de Clientes / Passwords	PostgreSQL (clientes, usuarios)	Snapshot básico desnormalizado	**No viajan por seguridad y privacidad**

3.b Plan de Mantenimiento Integral (Ampliación de la Unidad 6)

El plan de contingencia de la Unidad 6 se amplía para abarcar la infraestructura multi-motor y los dispositivos móviles:

Motor	¿Qué se respalda?	Frecuencia y Herramienta	¿Cómo se prueba la restauración?
PostgreSQL 17	Base completa y roles globales.	Diario a las 02:00 AM con pg_dump -Fc y pg_dumpall --globals-only.	Restauración automatizada en foodstore_restore_test validando 9 métricas de integridad (RTO $\le$ 15 min, RPO $\le$ 24 h).
MongoDB	Colección documental pedidos.	Semanal con mongodump --gzip (no crítico; reconstruible vía ETL).	Restauración en base de ensayo mongorestore comprobando conteo de documentos y coherencia de subtotales.
SQLite 3 (Tablet)	Exclusivamente borradores y cola de sincronización.	Al cierre de cada jornada con el comando online .backup de SQLite CLI.	Verificación estructural con PRAGMA integrity_check; (debe devolver 'ok') y conteo de solicitudes pendientes.

Comportamiento de las Tablets si se Restaura un Respaldo Viejo de PostgreSQL

Si el servidor central sufre una pérdida de datos y se restaura un backup con horas de desfase, las tablets pueden contener un catálogo más reciente que el servidor restaurado. Debido a que el servidor manda, las tablets aplican este protocolo:

1. **Invalidación Inmediata de Catálogo:** La tablet descarta su caché y descarga la instantánea fresca emitida por el servidor restaurado, sobreescribiendo producto_cache.
2. **Reevaluación de Solicitudes Pendientes:** El semáforo vuelve a evaluar las órdenes no enviadas. Si un precio retrocedió a su valor anterior, pasa a CONFLICTO y se informa al cliente.
3. **Reenvío Seguro con Idempotencia:** Las solicitudes despachadas con posterioridad al backup se reenvían con su mismo UUID. Al no existir en el backup restaurado, el servidor las procesa en orden sin duplicar ventas.

Periodicidad y Responsables de la Revisión de Conflictos y Rechazos

• **CONFLICTO (Amarillo):** Resuelto en tiempo real por el *vendedor en mostrador* con el cliente presente. Si el cliente acepta el precio actualizado, se regenera la comanda (Caso C2).
• **RECHAZADO (Rojo):** Revisado al final del turno por el *encargado del food truck* para anular borradores fallidos o pedir reposición de stock al centro de distribución.
• **Auditoría Semanal del DBA:** El *Administrador de Base de Datos* ejecuta la consulta de agregación para detectar productos con rechazos sistemáticos por stock insuficiente, ajustando los intervalos de sincronización.

3.c Revisión Integral de Código con IA (Code Review de la Cursada)

Se sometieron a auditoría estática de código asistida por IA la totalidad de scripts desarrollados a lo largo de la cursada (TP2 a TP10). A continuación se resumen los 10 hallazgos más relevantes con su respectiva decisión técnica fundamentada:

ID	Ubicación / Módulo	Hallazgo Identificado por la IA	Decisión	Motivo y Justificación Técnica
F01	TP2 · restricciones.sql	El trigger de stock lee sin FOR UPDATE: riesgo de sobreventa concurrente.	Aceptado	Condición de carrera real; mejora registrada para concurrencia.
F02	TP5 · views / indices	DROP CASCADE sin verificar contexto de base activa.	Aceptado	Práctica mejorable, aunque los scripts asumen base preconfigurada.
F03	TP7 · fn_resetear_pass	rol_soporte puede resetear claves de usuarios administradores.	Aceptado	Vulnerabilidad de privilegios; requiere cláusula WHERE rol != 'ADMIN'.
F04	TP7 · fn_autenticar	Discrepancia en el uso de btrim() en contraseñas entre scripts.	Aceptado	Riesgo menor en contraseñas con espacios iniciales o finales.
F05	TP2 · schema / triggers	Un pedido ENTREGADO se puede borrar físicamente arrastrando su detalle.	**Aceptado**	**Comprobado en base de datos de laboratorio (Ver detalle en pág. 6).**
F06	TP9 · prueba_val_expr	Validador $expr controla aritmética pero omite rangos de tipos.	Aceptado	Correcto: requiere combinar $jsonSchema con expresiones $expr.
F07	TP10 · recargar_snap	DELETE FROM producto_cache rompe claves foráneas con borradores activos.	**Corregido**	**Comprobado y corregido mediante sentencia UPSERT (Ver detalle en pág. 6).**
F08	TP10 · importar_snap	CAST de texto no numérico en SQLite produce 0 de forma silenciosa.	Aceptado	Riesgo bajo: el CSV proviene de columnas bigint tipadas de PostgreSQL.
F09	TP10 · recargar_snap	Afirma que un producto dado de baja permanece disponible en la tablet.	**Descartado**	**Descartado con fundamento técnico y refutado (Ver detalle en pág. 6).**
F10	TP8 · backup_foodstore	Métricas de control medidas en transacción ajena a pg_dump.	Descartado	En reposo durante la ventana nocturna no hay escrituras concurrentes.

Criterio Metodológico de Clasificación:

• **Aceptado:** El hallazgo describe una debilidad real del código. En TPs anteriores ya calificados, queda asentado como lección aprendida sin alterar la entrega original.

• **Corregido:** El hallazgo fue identificado en la etapa de diseño de este trabajo práctico y se refactorizó activamente antes de la versión final.

• **Descartado con Fundamento:** La observación de la IA constituye una mala interpretación del modelo de negocio o fue desestimada mediante pruebas empíricas en el motor.

3.c Desarrollo en Profundidad de Tres Hallazgos Críticos

1. Hallazgo F05 (TP2) — Borrado físico de pedidos cerrados · [ACEPTADO]

Observación de la IA: La IA advirtió que el trigger de integridad de TP2 únicamente interceptaba eventos UPDATE. Dado que la clave foránea entre detalles_pedido y pedidos poseía la cláusula ON DELETE CASCADE, un usuario con privilegios podía ejecutar DELETE FROM pedidos WHERE id_pedido = 1;, eliminando permanentemente la venta y sus renglones sin disparar ningún error.
Comprobación Empírica en PostgreSQL 17: Dentro de un bloque BEGIN ... ROLLBACK, se verificó que un UPDATE sobre el pedido entregado 1 era rechazado por el trigger, pero un DELETE sobre el mismo pedido eliminaba la fila y purgaba sus 3 líneas de detalle. El hallazgo fue aceptado e incorporado como lección aprendida: las reglas de inmutabilidad deben aplicarse a eventos BEFORE DELETE OR UPDATE.

2. Hallazgo F07 (TP10) — La recarga destructiva rompe la tablet · [CORREGIDO]

Observación de la IA: Al recargar la instantánea del servidor en la tablet móvil, el diseño inicial contemplaba vaciar la caché con DELETE FROM producto_cache; antes de reinsertar los productos actualizados. La IA detectó que al estar activas las claves foráneas (PRAGMA foreign_keys = ON;), si existía un borrador abierto en item_borrador haciendo referencia a un producto de la caché, la recarga se abortaba con error de violación de clave foránea.
Comprobación y Corrección en SQLite: Se recreó el fallo en el simulador y se corrigió el script recargar_snapshot.sql implementando la cláusula **UPSERT** (INSERT INTO producto_cache ... ON CONFLICT (id) DO UPDATE SET ...). De este modo, las filas maestras se actualizan en su lugar sin borrarse, preservando la integridad de los borradores en comanda.

3. Hallazgo F09 (TP10) — Falso positivo sobre productos dados de baja · [DESCARTADO CON FUNDAMENTO]

Observación de la IA: La IA argumentó que: *"Al recargar mediante UPSERT, si un producto es eliminado en el servidor, no sufrirá UPDATE y continuará disponible para la venta en la tablet indefinidamente, provocando rechazos masivos en la feria"*.
Refutación Técnica y Comprobación en Laboratorio: La afirmación de la IA es **falsa y fue descartada con fundamento** por dos razones:
1. En Food Store la baja de productos es **lógica** (regla R7), no física. La vista v_snapshot_movil en PostgreSQL **sí exporta el producto inactivo**, enviándolo con los indicadores disponible = 'f' y eliminado = 't'.
2. Al ejecutarse el UPSERT en la tablet, el producto se actualiza con disponible = 0 y eliminado = 1. Inmediatamente la consulta offline del Caso C1 (WHERE disponible = 1 AND eliminado = 0) lo excluye de la pantalla del vendedor. Asimismo, si existía una solicitud previa, el semáforo la clasifica en **`RECHAZADO` (Rojo)** con el motivo 'Producto no disponible'. Se demostró que la IA asumió erróneamente que una baja de producto implicaba su desaparición del CSV.

Conclusión de la Capa de Datos: La integración de PostgreSQL como motor transaccional de registro (System of Record), MongoDB como almacén optimizado de lectura para historial de ventas, y SQLite como base local desconectada bajo el principio «El móvil propone, el servidor dispone», consolida una arquitectura de datos robusta, tolerante a fallas y 100% alineada a las mejores prácticas de la industria.